FT-CONTRACTS SECURITY REVIEW REPORT

SAW-MON & NATALIE
@SWONT

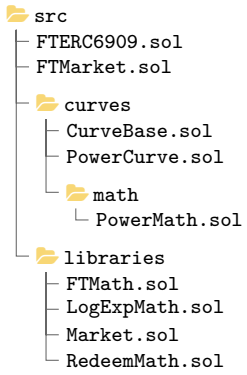
ABSTRACT. This report provides a security review of the ft-contracts repository. All issues have been analyzed and categorized by severity level.

CONTENTS

Date: January 8, 2026.

Executive Summary

Scope. This review was focused on math-related formulas and functionality of the files below in the `ft-contracts` repository:



Due to limited time of the review the modified Balancer v2 library `LogExpMath.sol` was not fully reviewed. Informal guidance has been provided to reduce the exposure of the current codebase to this library. Most of the review was focused on the power curve mechanism and its related mathematical formulas. During the review phase some out of scope findings have also been discovered that are shared in this report.

Below are the commit hashes reviewed, starting with the initial commit hash provided by the client and finally the fix review commit hash which includes all the fixes to the findings in this report.

Table 1. Security Issues by Severity

Severity	Count
Critical	0
High	2
Medium	7
Low	6
Informational	9
Other	2
Total	26

Disclaimer. This security assessment is provided for informational purposes only and reflects the reviewers' analysis of the specified codebase at a particular point in time, based solely on the materials, documentation, and representations made available during the engagement and within the agreed scope. The assessment applies exclusively to the specific commit, version, configuration, and deployment assumptions reviewed. Any modifications, extensions, integrations, parameter changes, or deployment in different environments may materially affect the security properties of the system and invalidate the conclusions of this report.

This assessment does not provide, and expressly disclaims, any warranties or guarantees of any kind, whether express or implied, including but not limited to warranties of merchantability, fitness for a particular purpose, correctness, completeness, or security. No representation is made that this review identifies all possible vulnerabilities, logic errors, economic risks, or attack vectors. The absence of reported findings does not imply that the code is free of defects or that it is secure against all current or future threats.

Security assessments are inherently limited in scope and depth and cannot be considered comprehensive. The results presented herein should not be relied upon as the sole measure of the system's security. Ongoing security practices—including additional independent reviews, continuous monitoring, testing, formal verification, and bug bounty programs—are strongly recommended.

Any recommendations or remediation guidance provided in this report, including example code or design suggestions, are illustrative in nature and are intended to convey general approaches rather than complete, tested, or production-ready solutions. Such recommendations may be incomplete or unsuitable for the client's specific implementation and should be independently evaluated, tested, and validated before adoption.

The contents of this report do not constitute legal, regulatory, tax, investment, financial, or other professional advice, and should not be relied upon as such. Nothing contained herein shall be construed as an endorsement, certification, approval, or guarantee of the reviewed project, protocol, or codebase.

To the maximum extent permitted by applicable law, the reviewers disclaim all liability for any direct, indirect, incidental, consequential, or special damages, losses, or claims arising out of or in connection with the use of this report, the reviewed code, or any reliance on the findings or recommendations contained herein. Responsibility for the secure development, deployment, operation, and maintenance of the system remains solely with the client.

H1. `ActionSimple` does not verify that the `market` input provided is a registered/valid one.

Tags. OUT OF SCOPE | SEVERITY: HIGH | LIKELIHOOD: HIGH | IMPACT: HIGH | CLIENT: FIXED | VERIFIED BY: SR

Context.

- ActionSimple.sol#L37
- ActionSimple.sol#L75
- ActionSimple.sol#L94

Description. The `market` parameter provided to:

- `swapSimple`
- `claimSimple`
- `claimAllSimple`

is provided by the `initiator`. There are no checks in the inherited child `FTRouter` contract to make sure this parameters is registered in the `controller`.

Recommendation. Make sure the provided `market` parameters are registered in `controller`.

Client. Fixed in 0f9efbc632b284f4bfabf72fb68ac6c98bc06996.

H2. Treasury fee should be added to the cost of increasing the outcome token supply when minting.

Tags. SEVERITY: HIGH | LIKELIHOOD: HIGH | IMPACT: HIGH | CLIENT: FIXED | VERIFIED BY: SR

Context.

- Market.sol#L118-L119
- Market.sol#L130
- PowerMath.sol#L30-L37

Description. When minting an outcome token, the caller would need to pay:

- cost of moving the along the `curve` by increasing the supply token for the outcode token
- treasury fee

These values should be added up to make sure the following approximate invariant holds (not considering the error terms and also the redeem tax):

$$A_{tot} \approx \sum_{oid} \frac{10^{d_{(T_C,pid)}}}{10^{18}} \left(\texttt{cost}_{\mathcal{C}}(s_{\mathcal{C}}^{start} + S_{tot}^{oid}) - \texttt{cost}_{\mathcal{C}}(s_{\mathcal{C}}^{start})\right)$$

In the current implementation, the treasury fee is taken from the collateral cost that would need to be paid to move along the curve (ie, the outcome token curve is being underpaid):

$$A_{tot} \xrightarrow{\text{mint}} A_{tot} + (\Delta a_{curve} - \Delta a_{tr})$$

One should instead have:

$$A_{tot} \xrightarrow{\text{mint}} A_{tot} + \Delta a_{curve}$$

Recommendation. Here is a rough draft of the changes that would need to be applied:

```
diff --git a/src/curves/math/PowerMath.sol b/src/curves/math/PowerMath.sol
index c48e6be..c678b43 100644
--- a/src/curves/math/PowerMath.sol
+++ b/src/curves/math/PowerMath.sol
@@ -29,12 +29,13 @@ library PowerMint {

         // NOTE: it cannot be -ve, a -ve area implies you get paid to buy outcomes
         // GOAL: maximize collateralFromUser => user pays highest estimated cost
-        collateralFromUser = 1;
+        uint256 collateralCostToMoveOnCurve = 1;
         if (collateralTo > collateralFrom) {
-            collateralFromUser = collateralTo - collateralFrom;
+            collateralCostToMoveOnCurve = collateralTo - collateralFrom;
         }
-        // treasury takes % from user, user does not pay extra for mints
-        collateralToTreasury = collateralFromUser.fullMulDiv(feeRate, FTMath.FT_ONE);
+        // treasury takes % from user, user DOES pay extra for mints
+        collateralToTreasury = collateralCostToMoveOnCurve.fullMulDivUp(feeRate, FTMath.FT_ONE);
+        collateralFromUser = collateralCostToMoveOnCurve + collateralToTreasury;
     }
```

Note

1. `PowerMint.guessOtDelta` and other inlined `PowerMint.calSwap` code snippets would need to be updated accordingly.
2. The decimal change for `collateralToTreasury` was **NOT** recommended to round up to keep the changes minimal and also make sure more collateral is accounted for the curve/pool instead of the treasury (in regards to the rounding error terms).
3. Potentially one could remove the `check` regarding `collateralToTreasury` to be bigger than `collateralDeltaIn` . But it is also ok to keep it for now.

Client. Fixed in `45db1fccd878a26b0da5ef814e5b6848f17387e2` .

Medium Severity Issues

M1. `PowerCurve.calMarginalPrice` , `simCost` and `simMarginalPrice` do not consider the `start` supply offset and return values have different precisions.

Tags. SEVERITY: MEDIUM | LIKELIHOOD: HIGH | IMPACT: MEDIUM | CLIENT: FIXED | VERIFIED BY: SR

Context.

- PowerCurve.sol#L59
- PowerCurve.sol#L164
- PowerCurve.sol#L170

Description.

1. When calculating the `calMarginalPrice` (`simCost` and `simMarginalPrice`), `PowerCurve` does not consider the `start` supply offset. This contracts the use of `start` when calculating the cost of `mint` / `redeem` of an outcome token supply.
2. `calMarginalPrice` 's returned precision is in collateral decimals, whereas `simCost` and `simMarginalPrice` use 18 decimal precisions.

Recommendation.

1. Apply the following change to use the `start` offset:

```
diff --git a/src/curves/PowerCurve.sol b/src/curves/PowerCurve.sol
index fd3a712..53edfa4 100644
--- a/src/curves/PowerCurve.sol
+++ b/src/curves/PowerCurve.sol
@@ -56,7 +56,7 @@ contract PowerCurve is IFTCurve {
         PowerMath.CurveParams memory curve = readCurve();
         MarketState memory state = readMarketState(market, tokenId);

-        price = curve.calMarginalPrice(state.otCurrent);
+        price = curve.calMarginalPrice(state.otCurrent + curve.start);

         uint256 collateralDecimals = IFTMarket(market).collateralDecimals();
         price = price.fullMulDiv(10 ** collateralDecimals, FTMath.FT_ONE);
@@ -161,13 +161,13 @@ contract PowerCurve is IFTCurve {
     /// @inheritdoc IFTCurve
     function simCost(uint256 otSupply) external view returns (uint256 cost) {
         PowerMath.CurveParams memory curve = readCurve();
-        return curve.calCost(otSupply);
+        return curve.calCost(otSupply + curve.start);
     }

     /// @inheritdoc IFTCurve
     function simMarginalPrice(uint256 otSupply) external view returns (uint256 price) {
         PowerMath.CurveParams memory curve = readCurve();
-        return curve.calMarginalPrice(otSupply);
+        return curve.calMarginalPrice(otSupply + curve.start);
     }

     /// @inheritdoc IFTCurve
```

2. If required `simCost` and `simMarginalPrice` should also return in collateral decimal precision. Otherwise the precision requirement for these functions should be documented in `IFTCurve`

Client. Supply start offsets have been corrected internally in `PowerMath` library.

Fixed in `6f79f8d71263c218764ea71c04900dca478cbfc0`

M2. Inaccurate `PowerMint.guessOtDelta` Binary Search.

Tags. SEVERITY: MEDIUM | LIKELIHOOD: MEDIUM | IMPACT: MEDIUM | CLIENT: FIXED | VERIFIED BY: SR

Context.

- PowerMath.sol#L67-L71
- PowerMath.sol#L99
- PowerMath.sol#L110
- PowerMath.sol#L58
- PowerMath.sol#L102

Description. `PowerMint.guessOtDelta` starts its initial guess with s_g^{off} and returns early if:

$$C(s_g^{off}) : \left\lfloor \frac{10^{18}-\epsilon}{10^{18}} \cdot \Delta a_c \right\rfloor \leq \max\left(0, \texttt{cost}_{\mathcal{C}}^{\uparrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid} + s_g^{off}) - \texttt{cost}_{\mathcal{C}}^{\downarrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid})\right) \leq \Delta a_c$$

If the above condition fails, the algorithm continues and sets both:

$$s_g^{min} = s_g^{max} = \max(1, s_g^{off})$$

In the next loop it tries to increment s_g^{max} so that:

$$\Delta a_c \leq \max\left(0, \texttt{cost}_{\mathcal{C}}^{\uparrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid} + s_g^{max}) - \texttt{cost}_{\mathcal{C}}^{\downarrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid})\right)$$

In the loop after it performs a binary search using:

$$s_g^{mid} = \left\lceil \frac{s_g^{min} + s_g^{max}}{2} \right\rceil$$

until the condition for $C(s_g^{mid})$ would satisfy. Until the condition is satisfied depending on the value of:

$$\max\left(0, \texttt{cost}_{\mathcal{C}}^{\uparrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid} + s_g^{mid}) - \texttt{cost}_{\mathcal{C}}^{\downarrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid})\right)$$

compared to Δa_c either one of the $s_g^{min/max}$ is updated.

There are a few problems with the current implementation of the above algorithm:

1. The `initial values picked` for $s_g^{min/max}$ might not satisfy the desired values regarding the required cost for these delta outcome token shares
2. `collateralDeltaRequired` is not set to 0 when `collateralToGuess <= collateralFrom` and thus can alter the calculations for the branches of the next iteration.
3. The s_g^{max} `update` is not symmetric compared to the s_g^{min} update and the following invariant might not be guaranteed with this update:

$$\Delta a_c \leq \max\left(0, \texttt{cost}_{\mathcal{C}}^{\uparrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid} + s_g^{max}) - \texttt{cost}_{\mathcal{C}}^{\downarrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid})\right)$$

4. The algorithm assumes the delta cost function used below is monotonic for its binary search algorithm, but it might be true in general.

$$\Delta\texttt{cost}_{\mathcal{C}}(s) = \max\left(0, \texttt{cost}_{\mathcal{C}}^{\uparrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid} + s) - \texttt{cost}_{\mathcal{C}}^{\downarrow}(s_{\mathcal{C}}^{start} + S_{tot}^{oid})\right)$$

5. The lower-bound threshold $\left\lfloor \frac{10^{18}-\epsilon}{10^{18}} \cdot \Delta a_c \right\rfloor$ might not be tight enough.

Recommendation.

1. When the initial guess fails, depending on whether $\Delta\texttt{cost}_{\mathcal{C}}(s)$ is bigger or smaller compared to Δa_c set either the s_g^{max} or s_g^{min} then try to find the other outcome token bound in the first loop:

$$\Delta\texttt{cost}_{\mathcal{C}}(s_g^{off}) < \Delta a_c \Rightarrow \begin{cases} s_g^{min} = \max(1, s_g^{off}) \\ s_g^{max} \leftarrow \text{iteratively try to find this value} \end{cases}$$

$$\Delta\texttt{cost}_{\mathcal{C}}(s_g^{off}) > \Delta a_c \Rightarrow \begin{cases} s_g^{min} \leftarrow \text{iteratively try to find this value} \\ s_g^{max} = \max(1, s_g^{off}) \end{cases}$$

One should also return early if by chance we would hit the $C(s)$ condition while trying to estimate the initial values for either $s_g^{min/max}$.

2. Make sure if `collateralDeltaRequired` does not get set in the `if` branch, then it should be set to 0.
3. Just like updating s_g^{min}, if $s_g^{mid} = s_g^{max}$ `break` (this makes sense since we either have $s_g^{max} = s_g^{min}$ or $s_g^{max} = s_g^{min} + 1$ and thus there are no middle values left to compare. This is conditional to fixing **item 1.**). And assign $s_g^{max} \leftarrow s_g^{mid}$ without decrementing 1. This would guarantee that in each iteration of the binary search loop we have:

$$\Delta\texttt{cost}_{\mathcal{C}}(s_g^{min}) < \left\lfloor \frac{10^{18}-\epsilon}{10^{18}} \cdot \Delta a_c \right\rfloor < \Delta a_c < \Delta\texttt{cost}_{\mathcal{C}}(s_g^{max})$$

4. The use of `LogExpMath.sol` functions need to be thoroughly investigated.
5. If a tighter bound is desired one should use $\left\lceil \frac{10^{18}-\epsilon}{10^{18}} \cdot \Delta a_c \right\rceil$ instead.

Client. Most of the recommendations above have been applied in `45db1fccd878a26b0da5ef814e5b6848f17387e2` . Moreover discrete `tick` supply sizes have been enforced to the supply grid to reduce the supply set.

Footnote.

1. `calMarginalPrice` is only used in mainly this algorithm and some other `view` functions. Thus not an integral part of the codebase. One could have use the forward difference of the cost function as the definition for the price function:

$$p(s) = c(s+1) - c(s)$$

2. The `first loop` is a rough approximation where the negative term in the discrete newton method is not used:

$$s \leftarrow s + \frac{\Delta a_c}{p(s)} - \frac{c(s) - c(s_0)}{p(s)}$$

M3. Overestimate the treasury fee when redeeming.

Tags. SEVERITY: MEDIUM | LIKELIHOOD: HIGH | IMPACT: MEDIUM | CLIENT: FIXED | VERIFIED BY: SR

Context.

- `PowerMath.sol#L147`

Description. When outcome tokens get redeemed in `PowerRedeem.calSwap` the `collateralToTreasury` is underestimated:

$$\Delta a_c^{tr} = \left\lfloor \frac{r_{fee}}{10^{18}} \Delta a_c^{pool} \right\rfloor$$

Recommendation. To favour the protocol make sure to round up when calculating the `collateralToTreasury` in the redeem route using `fullMulDivUp` .

Client. Fixed in `59d149f260be06d19b3d414b4b42609599f1349f` .

Footnote. One could also overestimate the `collateralToTreasury` in `PowerMint.calSwap` to favour the treasury versus the pool. But the current implementation favouring the market collateral pool also makes sense.

M4. `PowerRedeem.guessOtDelta` binary search inconsistencies.

Tags. SEVERITY: MEDIUM | LIKELIHOOD: HIGH | IMPACT: MEDIUM | CLIENT: FIXED | VERIFIED BY: SR

Context.

- `PowerMath.sol#L167`
- `PowerMath.sol#L184`
- `PowerMath.sol#L176-L178`
- `PowerMath.sol#L192`

Description/Recommendation. Although `PowerRedeem.guessOtDelta` does not exactly mirror the implementation of `PowerMint.guessOtDelta` , it still shares some of the short comes and a few other inconsistencies.

1. The answer for the `otDeltaGuess` and `collateralDeltaReturned` is only accepted if one falls approximately close to (from below) `collateralDeltaTarget` . In the redeem route one should instead search the close approximation from above or both sides. Let:

$$f(\Delta s_{oid}) = \left\lfloor \left(\frac{10^{18} - r_{tax}(s^{\mathcal{C}}_{start} + S^{oid}_{tot}, \Delta s_{oid})}{10^{18}} \right) \max\left(0, \texttt{cost}^{\downarrow}_{\mathcal{C}}(s^{\mathcal{C}}_{start} + S^{oid}_{tot}) - \texttt{cost}^{\uparrow}_{\mathcal{C}}(s^{\mathcal{C}}_{start} + S^{oid}_{tot} - \Delta s_{oid})\right) \right\rfloor - \text{fee term}$$

and so one is searching for Δs_{oid} such that:

$$f(\Delta s_{oid}) \in \left[\left\lfloor \frac{10^{18} - \epsilon}{10^{18}} \cdot \Delta a_c \right\rfloor, \Delta a_c \right]$$

instead one should perform the search in either:

$$f(\Delta s_{oid}) \in \left[\Delta a_c, \left\lfloor \frac{10^{18} + \epsilon}{10^{18}} \cdot \Delta a_c \right\rfloor \right]$$

or

$$f(\Delta s_{oid}) \in \left[\left\lfloor \frac{10^{18} - \epsilon}{10^{18}} \cdot \Delta a_c \right\rfloor, \left\lfloor \frac{10^{18} + \epsilon}{10^{18}} \cdot \Delta a_c \right\rfloor \right]$$

The user can set a max allowed outcome token to be burnt/transferred in a router implementation.

2. The provided s_g^{off} `otDeltaGuessOffchain` can fall in the accepted range defined in **item 1.** or if not then the values of $s_g^{min/max}$ can be set accordingly by checking on which side of the accepted range the value of $f(s_g^{off})$ falls. If above the range one can set the value of s_g^{max} to s_g^{off} and try to estimate a value for s_g^{min} (just like the recommendation for `PowerMint.guessOtDelta` in finding #15) and if the other way around one would set $s_g^{min} = s_g^{off}$ and try to find a potential value for s_g^{max}
3. In the binary search loop one should keep the invariant that $f(s_g^{min})$ is below the accepted range and $f(s_g^{max})$ is above the accepted range. Thus the update for `guess.otGuessMax` should be changed exactly like the branch for `guess.otGuessMin`
4. The binary search relies on the assumption that $f(s)$ is a monotonic function but this might not be true in a range one is interested in due to the current usage of `LogExpMath` library. Therefore potential solution to the optimization problem might be missed or one might not perform an ideal search algorithm.

Client. Most of the recommendations have been applied in `45db1fccd878a26b0da5ef814e5b6848f17387e2` . Regarding **item 1.**, the current search criteria is kept due to design choice.

M5. `LogExpMath` diff/modifications.

Tags. OUT OF SCOPE | SEVERITY: MEDIUM | LIKELIHOOD: MEDIUM | IMPACT: HIGH | CLIENT: FIXED | VERIFIED BY: SR

Description. Current `Balancer v2` implementation and the implementation used in this protocol have the following rough changes:

1. `ft-contracts` adds `unchecked` blocks to all util functions.
2. `log(int256 arg, int256 base)` is removed in `ft-contracts`
3. `SPDX-License-Identifier` has been changed
4. `solc` version pragma has been changed
5. `require` statements have been modified to not use error codes but instead error strings.
6. `UONE_18` has been added to be used in `PowerMath.sol`
7. The placement order of the util functions in the library has been rearranged. `pow` has been moved to the top.
8. the condition `x >> 255 == 0` has been changed to `x < 2 ** 255` (`LogExpMath.sol#L267`)
9. typos: `Intrnal` (`LogExpMath.sol#L440`) and `r`esult` (`LogExpMath.sol#L263`)

Recommendation.

1. Do not use `unchecked` blocks unless proof is provided that these `unchecked` blocks are safe.
2. It would be best to keep the order of the util functions the same.
3. `x >> 255 == 0` might be cheaper to use and also it might be best to use the same conditional statement as Balancer v2 implementation to keep the changes minimal.
4. Types need to be fixed.
5. Provide link and commit hash of the Balancer v2 implementation used in the NatSpec of `LogExpMath.sol` .

Client. Recommendation from the footnote has been applied:

- `1e509d726648f15378a03e8351991e4024830e99` copies balancer implementation (with 2 minor formatting adjustments)
- `5c4361bb369876bd615385f6ca8f0a8d820b20a0` applies minor changes:
  - re-writing `require` statements
  - introducing a new constant `UONE_18`
  - changing the `solc version pragma`

Footnote. In general, it would have been best to

- copy the implementation from Balancer to the protocol
- commit to this addition
- apply the changes and commit again

This way the modification would have been more transparent.

M6. `calGrowthFactor` does not consider the `start` supply offset for the numerator.

Tags. SEVERITY: MEDIUM | LIKELIHOOD: HIGH | IMPACT: HIGH | CLIENT: FIXED | VERIFIED BY: SR

Context.

- PowerMath.sol#L145
- RedeemMath.sol#L43

Description. The redeem growth factor is calculated as:

$$f_g(\Delta s_{oid}, s^{\mathcal{C}}_{start} + S^{oid}_{tot}) = \left\lceil \frac{c^g_1}{10^{18}} \cdot \texttt{exp}\left(\left\lceil \frac{c^g_2}{10^{18}} \cdot \min(10^{10}, \left\lceil \frac{\Delta s_{oid}}{s^{\mathcal{C}}_{start} + S^{oid}_{tot}} \cdot 10^{18} \right\rceil)\right\rceil\right)\right\rceil$$

one can see that:

$$\left\lceil \frac{c^g_1}{10^{18}} \cdot \texttt{exp}\left(\frac{c^g_2}{10^{8}}\right)\right\rceil \leq f_g(\cdots) \leq \left\lceil \frac{c^g_1}{10^{18}} \cdot \texttt{exp}(c^g_2)\right\rceil$$

But since the `start` curve offset $s^{\mathcal{C}}_{start}$ does not get added to the numerator of the inner fraction $\frac{\Delta s_{oid}}{s^{\mathcal{C}}_{start}+S^{oid}_{tot}}$, one might not be able to enforce/reach the upper-bound for the growth factor even when $\Delta s_{oid} = S^{oid}_{tot}$ (`otFrom == otDelta`). One should instead calculate and use:

$$f_g(s^{\mathcal{C}}_{start} + \Delta s_{oid}, s^{\mathcal{C}}_{start} + S^{oid}_{tot})$$

which should impose a stricter redeem tax rate through the growth factor.

Recommendation. Apply the following changes:

```
diff --git a/src/curves/math/PowerMath.sol b/src/curves/math/PowerMath.sol
index c48e6be..4515c14 100644
--- a/src/curves/math/PowerMath.sol
+++ b/src/curves/math/PowerMath.sol
@@ -142,7 +142,7 @@ library PowerRedeem {
             collateralTotal = collateralFrom - collateralTo;
         }

-        uint256 taxRate = RedeemMath.calRedeemTaxRate(redeem, otFrom, otDelta);
+        uint256 taxRate = RedeemMath.calRedeemTaxRate(redeem, otFrom, otDelta + curve.start);
         uint256 collateralFromPool = collateralTotal.fullMulDiv(FTMath.FT_ONE - taxRate, FTMath.FT_ONE);
         collateralToTreasury = collateralFromPool.fullMulDiv(feeRate, FTMath.FT_ONE);
         collateralToUser = collateralFromPool - collateralToTreasury;
```

Client. Fixed in `45db1fccd878a26b0da5ef814e5b6848f17387e2` . Start supply offset gets applied to both `otFrom` and `otDelta` .

M7. Instantaneous mint and redeem including perturbation.

Tags. SEVERITY: MEDIUM | LIKELIHOOD: MEDIUM | LIKELIHOOD: HIGH | CLIENT: ACKNOWLEDGED

Context.

- PowerCurve.sol

Description. If a user would mint Δs and then redeem the same share atomically, the delta collateral paid to the curve/pool becomes (not considering the fees and the redeem tax):

$$\begin{aligned}\Delta a_c = &+ \max(1, \texttt{cost}^{\uparrow}_{\mathcal{C}}(S + \Delta s) - \texttt{cost}^{\downarrow}_{\mathcal{C}}(S)) \\ &- \max(0, \texttt{cost}^{\downarrow}_{\mathcal{C}}(S + \Delta s) - \texttt{cost}^{\uparrow}_{\mathcal{C}}(S)) \\ &\in \{0, 1, 2\}\end{aligned}$$

The above calculation is based on calculations and recommendations in:

- finding H2
- finding I5

Furthermore, if one would apply:

- treasury fees
- redeem tax rate
- outcome token to collateral token precision change

The value of Δa_c would stay non-negative. Thus the instantaneous mint/redeem strategy for a user would incur loss (user would not be able to extract value).

One can show above is stronger than the current and modified check in the mint route for example (but it is recommended to keep these checks in place):

- https://github.com/Saw-mon-and-Natalie/review-ft-contracts/issues/8

For the reverse redeem/mint strategy, one can also show that using the same amount of collateral received from redeem, one would not be able to mint more shares than the amount of shares redeemed (ie no free boosting up the position). Unless due to perturbation of the amount to mint we would get:

$$\begin{aligned}&+\max(0, \texttt{cost}^{\downarrow}_{\mathcal{C}}(S+\Delta s) - \texttt{cost}^{\uparrow}_{\mathcal{C}}(S))\\&-\max(1, \texttt{cost}^{\uparrow}_{\mathcal{C}}(S+\Delta s+\epsilon_s) - \texttt{cost}^{\downarrow}_{\mathcal{C}}(S))\\&\geq 0\end{aligned}$$

Simplifying this by assuming max values are greater than 1 we get:

$$0 \leq \texttt{cost}^{\downarrow}_{\mathcal{C}}(S+\Delta s) - \texttt{cost}^{\uparrow}_{\mathcal{C}}(S+\Delta s+\epsilon_s) + \underbrace{\texttt{cost}^{\downarrow}_{\mathcal{C}}(S) - \texttt{cost}^{\uparrow}_{\mathcal{C}}(S)}_{\in\{-1,0\}}$$

Thus the question becomes whether there exists some variables such that:

$$\begin{aligned}\underbrace{\texttt{cost}^{\uparrow}_{\mathcal{C}}(S) - \texttt{cost}^{\downarrow}_{\mathcal{C}}(S)}_{\in\{0,1\}} &\leq \texttt{cost}^{\downarrow}_{\mathcal{C}}(S) - \texttt{cost}^{\uparrow}_{\mathcal{C}}(S+\epsilon_s)\\&= \left\lfloor \frac{10^{18}}{c_3}\texttt{pow}(S, 10^{18}+c_1)\right\rfloor - \left\lceil \frac{10^{18}}{c_3}\texttt{pow}(S+\epsilon_s, 10^{18}+c_1)\right\rceil\\&= \frac{10^{18}}{c_3}\left(\texttt{pow}(S, 10^{18}+c_1) - \texttt{pow}(S+\epsilon_s, 10^{18}+c_1)\right) - \underbrace{\epsilon_1}_{\in[0,2)}\end{aligned}$$

A stricter (weaker with a smaller domain) search criteria would be, if one could find parameters such that:

$$\frac{3\cdot c_3}{10^{18}} \leq \texttt{pow}(S, 10^{18}+c_1) - \texttt{pow}(S+\epsilon_s, 10^{18}+c_1)$$

If (c_1, c_2) were of the form $(\lfloor \frac{n}{n+1}10^{18}\rceil, 10^{6+18})$ where $n \in \mathbb{N}$ we would get:

$$3\cdot 10^6 \cdot \frac{10^{18} + \lfloor \frac{n}{n+1}10^{18}\rceil}{10^{18}} \leq \texttt{pow}(S, 10^{18} + \lfloor \frac{n}{n+1}10^{18}\rceil) - \texttt{pow}(S+\epsilon_s, 10^{18} + \lfloor \frac{n}{n+1}10^{18}\rceil)$$

In general fixing the exponent n a stronger guarantee would be to prove/show that $\texttt{pow}(s, n)$ is a non-decreasing function with respect to the base parameter s. This would also prove that perturbing the mint/redeem strategy would not be value-extracting.

Recommendation. Prove/show that the $\texttt{pow}(\cdot, n)$ function is non-decreasing and if not use a replacement function that is non-decreasing and has the other desired properties.

Client. Acknowledged. It's a bit complex to prove the taylor series mathematically so we have created a `script` that brute forces for cost function.

We have so far checked for

$$\Delta s_{tick} = 10^{18}$$

$$s_{start} \approx 8.89 \cdot 10^{18}$$

for

$$[s_{min}, s_{max}] = [0, 3\cdot 10^{26}]$$

Low Severity Issues

L1. `IFTCurve` 's `calMintCostByOtDelta` and `calRedeemValueByOtDelta` can potentially modify state.

Tags. OUT OF SCOPE | SEVERITY: LOW | LIKELIHOOD: HIGH | IMPACT: LOW | CLIENT: FIXED | VERIFIED BY: SR

Context.

- IFTCurve.sol#L11-L27

Description. The current implementations of `IFTCurve` have the `view` state mutability for the functions below:

- `calMintCostByOtDelta`
- `calRedeemValueByOtDelta`

Moreover, other contracts such as `MarketFactory` assumes that calls to these functions with the same input parameters should return the same output values. To enforce this invariant even more, it would be best to declare these functions as `view` functions at the `Interface` level so that the markets would perform `staticcall` s instead of potentially state-modifying `call` s.

Recommendation. Make sure these 2 functions are marked as `view` functions in `IFTCurve`. If they are planned to be used with state-modifying curves for some potential future curves, this intent should be documented in the NatSpecs.

Client. NatSpec has been added in `9ecd7eeaba14fda4bf4fc3125b09b44cebf324ee`.

L2. Users can transfer collateral or outcomes tokens to the market.

Tags. SEVERITY: LOW | LIKELIHOOD: HIGH | IMPACT: LOW | CLIENT: FIXED | VERIFIED BY: SR

Context.

- FTMarket.sol#L122
- FTMarket.sol#L152
- FTERC6909.sol#L90-L108

Description. In the context of `mintCollateralToExactOt` and `redeemExactOtToCollateral`:

- minting outcome tokens to the market itself is blocked
- redeeming collateral token to the market itself is blocked

`FTMarket` inherits from `FTERC6909` which allows direct transfers or outcome tokens to any non-zero non-self addresses. So:

1. A user can directly transfer an outcome token of a market M_0 to the same market
2. A user can directly transfer an outcome token T_o of a market M_0 to a different market M_1. The receiver market M_1 might or might not be using T_o as a collateral. (intra-market direct transfers)
3. A user can directly transfer an `IERC20` collateral (or any other unrelated token) to a market M_0. This would be out of the scope of any implementation, since the protocol would not be able to block these transfers.

Recommendation.

1. `transfer` and `transferFrom` should be modified or overwritten to block transfers to the market itself.
2. `transfer` and `transferFrom` should be modified or overwritten to block transfers to any market registered in `FTMarketController` by calling `isMarket(to)` to make sure `to` is not a registered market unless the caller to `transfer{From}` is a registered market. In nutshell, blocking external entities to direct transfer outcomes tokens of a registered market to another market.
3. It can only be documented and the protocol would need to keep an internal track of its own token transfers using storage parameters.

Client. Items 1. and 2. have been fixed in `991572113f4f38268072076a737f856a2bf625f1`.

Footnote.

1. These tokens can be skimmed out to the treasury.

L3. `PowerMint.guessOtDelta` does not conform to the `PowerMint.calSwap` logic.

Tags. SEVERITY: LOW | LIKELIHOOD: LOW | IMPACT: MEDIUM | CLIENT: FIXED | VERIFIED BY: SR

Context.

- PowerMath.sol#L32
- PowerMath.sol#L48
- PowerMath.sol#L55-L57
- PowerMath.sol#L98-L100

Description. `PowerMint.calSwap(...)` clamps `collateralDeltaRequired` from below to 1. But this clamping is not performed in `PowerMint.guessOtDelta` where the logic of `PowerMint.calSwap(...)` is inlined.

Recommendation. Make sure the value of `collateralDeltaRequired` is clamped below by 1 in

- the initial off-chain guess check
- in each iteration of the binary search loop

Client. In `45db1fccd878a26b0da5ef814e5b6848f17387e2`, the `PowerMint.guessOtDelta` calls `PowerMint.calSwap` directly thus avoiding any diverging implementation.

Footnote. `PowerMint.guessOtDelta` inlines the logic of `PowerMint.calSwap(...)` whereas `PowerRedeem.guessOtDelta` calls `PowerRedeem.calSwap` thus avoiding any inconsistencies.

L4. `transferFromInitiator` does not consider `IERC6909` tokens when fetching balance.

Tags. OUT OF SCOPE | SEVERITY: LOW | LIKELIHOOD: MEDIUM | IMPACT: MEDIUM | CLIENT: FIXED | VERIFIED BY: SR

Context.

- ActionFromInitiator.sol#L34

Description. The fetch logic for `transferFromInitiator` is copied from `erc20TransferFromInitiator` which only includes the case for `IERC20` tokens. But `transferFromInitiator` could have a non-zero `tokenId` .

Recommendation. If `tokenId` is non-zero, one should fetch the balance using `IERC6909.balanceOf` which needs the owner and `tokenId` as an input.

Client. Fixed in `ab0b1031585693f4a65f5765253e24b079af6733` .

L5. The callback functions for redeem and claim are redundant.

Tags. SEVERITY: LOW | LIKELIHOOD: HIGH | IMPACT: LOW | CLIENT: FIXED | VERIFIED BY: SR

Context.

- IFTCallback.sol#L18-L35
- FTMarket.sol#L279-L282
- FTMarket.sol#L188-L190

Description. The information provided to `onRedeem` and `onClaim` are either:

- provided by the `caller` or
- returned by the called endpoint

Thus if necessary, the `caller` can perform its required operations after the call to `redeemExactOtToCollateral` or `claim` .

In the `redeemExactOtToCollateral` flow, the callback could cause the caller to use its to be burnt outcome tokens in another market (cross-market reentrancy) to perform some atomic transactions. It is preferable to avoid scenarios like this.

Recommendation. Remove the `onRedeem` and `onClaim` callbacks.

Client. Fixed in `8e14b5f4726dd8609309f9456a50af02bdffb64a` .

Footnote. The use of `onMint` callback makes sense, since in the callback function the `caller` can rebalance its supply of tokens and also provide the required token approvals.

L6. The order of operations in `_redeemOtToCollateral` should be updated.

Tags. SEVERITY: LOW | LIKELIHOOD: LOW | IMPACT: LOW | CLIENT: FIXED | VERIFIED BY: SR

Context.

- FTMarket.sol#L272-L283

Description. The order of operations in `_redeemOtToCollateral` should be updated. Collateral token should be transferred out after updating storage parameters.

Recommendation. The following rough patch reorders the operations so that collateral tokens would transfer out after updating storage parameters (moreover the callback has been removed):

```
diff --git a/src/FTMarket.sol b/src/FTMarket.sol
index 41f4c30..53ee65f 100644
--- a/src/FTMarket.sol
+++ b/src/FTMarket.sol
@@ -271,17 +271,13 @@ contract FTMarket is FTERC6909, IFTMarket, TokenHelper, ReentrancyGuard, HasFTEv

         _writeState(market);

-        _transferOut(collateral, parentTokenId, receiver, collateralToUser);
-        _transferOut(collateral, parentTokenId, market.treasury, collateralToTreasury);
-
         emit RedeemSwap(caller, receiver, tokenId, collateralToUser, otDeltaIn, collateralToTreasury);

-        // note: you can't redeem to trigger a callback to redeem the same thing again that is very dangerous!
-        if (dataCallback.length > 0) {
-            IFTRedeemCallback(caller).onRedeem(otDeltaIn, collateralToUser, dataCallback);
-        }
         _burn(caller, tokenId, otDeltaIn);

+        _transferOut(collateral, parentTokenId, receiver, collateralToUser);
+        _transferOut(collateral, parentTokenId, market.treasury, collateralToTreasury);
+
         // final check: mint cost >= redeem value
         (uint256 collateralMintCost,) = market.curve.calMintCostByOtDelta(market.market, tokenId, otDeltaIn, dataSwap);
         if (collateralMintCost < (collateralToUser + collateralToTreasury)) {
```

Client. Fixed in `8e14b5f4726dd8609309f9456a50af02bdffb64a` .

Informational Severity Issues

I1. Minor issues.

Tags. SEVERITY: INFORMATIONAL | CLIENT: FIXED | VERIFIED BY: SR

Description/Recommendation.

- `Market.sol#L43`: `NUM_OUTCOMES_MAX` is not used. If not planning to use this value to enforce an upper bound it can be removed from the library.
- `Market.sol#L44`: `FULL_BIPS` is not used.
- `FTERC6909.sol#L54-L56`: `FTERC6909.supportsInterface(...)` does not check against `IERC6909Metadata` and `IERC6909TokenSupply` .
- `TokenHelper.sol#L105`, `TokenHelper.sol#L111`: For type and typo safety use `abi.encodeCall` instead of `abi.encodeWithSelector`
- `ActionSimple.sol#L42`: The `otGuessMin` and `otGuessMax` provided by the `dataGuess` are not used and instead they are populated in `PowerMath.sol` . These fields can be omitted from the `calldata` and be added to the memory structure in `decodeGuessParam(...)` to default values of `0` .
- `Decoder.sol#L51`: The `guess.eps` provided by the user has only been checked to be non-zero. There are not explicit upper-bound checks performed during decoding. But when used there is an implicit check in `isASmallerApproxB` that would cause a revert if `guess.eps > FT_ONE` . This implicit check can be performed explicitly in `decodeGuessParam` to make sure `guess.eps <= FT_ONE` . One might also consider a harder upper-bound for this value.
- `Decoder.sol#L51`: `guess.eps == 0` could be allowed, even though the probability of finding exact solutions might be vey low.
- `Decoder.sol#L50`: `guess.maxIterations == 0` can be allowed. It would mean that the initial off-chain guess provided should hit close to the expected cost. Note that this value can also be clamped from above by `256` (due to using `uint256` and the binary-search).

Client. Fixed in `6f79f8d71263c218764ea71c04900dca478cbfc0`. Last item has been acknowledged.

I2. Restrictions are not applied to `IERC6909` collaterals.

Tags. OUT OF SCOPE | SEVERITY: INFORMATIONAL | LIKELIHOOD: LOW | IMPACT: LOW | CLIENT: FIXED | VERIFIED BY: SR

Context.

- `FTMarketController.sol#L64-L65`

Description. When a market is deployed using `deployMarketAndSeedLiquidity` , the combination of `(collateral, parentTokenId)` (T_C, o) can be:

1. $(T_C, 0)$ for `IERC20` type collaterals
2. (T_C, o) where $o \neq 0$ for `IERC6909` type collateral. There is no restrictions that this collateral type should be a combination of a registered market and one of its outcome token ids ($o \in \{1, 2, \cdots, 2^{n-1}\}$). Any so any `IERC6909` sub-token can be used.

Recommendation. Regarding **item 2.** either document that general `IERC6909` sub-token can be used or add on-chain checks to make sure previously registered market and one of its outcome token ids is passed to `deployMarketAndSeedLiquidity` .

Client. Fixed in `6f79f8d71263c218764ea71c04900dca478cbfc0`.

I3. `simPayout` does not consider of the total winning supply to be zero.

Tags. SEVERITY: INFORMATIONAL | CLIENT: FIXED | VERIFIED BY: SR

Context.

- `FTMarket.sol#L369-L380`

Description. `simPayout` does not consider the case of the total winning supply to be zero. The currently implementation would revert when `fullMulDiv` is called. This is unlike the `Market.claim(...)` utility function which has a special branch for the total winning supply to be zero.

Recommendation. Make sure `simPayout` handles the `otSupplyWinning == 0` case just like `Market.claim(...)` by returning `0` for `payout` .

Client. Fixed in `d2fe902f8efca54549b9bbd4a555cd635c26b34c`.

I4. Redeem tax applied does not follow the provided specifications.

Tags. SEVERITY: INFORMATIONAL | CLIENT: FIXED | VERIFIED BY: SR

Context.

- PowerMath.sol#L145

Description. In the provided internal specifications the redeem tax is calculated using:

$$r_{spec}(x_0, X, t_{passed}) = \int_{X-x_0}^{X} p(x) \cdot g(x, X) \cdot t(t_{passed}) \cdot s(x) dx$$

where as in the implementation one is using:

$$r_{impl}(x_0, X, t_{passed}) = g(x_0, X) \cdot t(t_{passed}) \cdot s(X) \cdot \int_{X-x_0}^{X} p(x) dx$$

One can show:

$$r_{spec}(x_0, X, t_{passed}) \leq r_{impl}(x_0, X, t_{passed})$$

Thus the implementation enforces a stricter tax rate which would be in the favour of the protocol.

Recommendation. Implementing the original formula from the specifications would be hard gas-intensive task. Perhaps the specs need to be updated to confirm with the current formula used in the implementation.

Client. The documentation has been updated to reflect the suggested formula:

$$r_{impl}(x_0, X, t_{passed}) = g(x_0, X) \cdot t(t_{passed}) \cdot s(X) \cdot \int_{X-x_0}^{X} p(x) dx$$

Footnote.

1. The value of $r_{impl}(x_0, X, t_{passed})$ is also clamped in the range $[\frac{1}{1000}, 1]$ which is not mentioned in the documentations.
2. The formula in the documentations have not been written accurately, perhaps the original intended integral was:

$$r_{spec,2}(x_0, X, t_{passed}) = \int_{X-x_0}^{X} p(x) \cdot g(x - (X - x_0), x) \cdot t(t_{passed}) \cdot s(x) dx$$

I5. The effects of power curve parameters on splitting the mint supply into batches.

Tags. SEVERITY: INFORMATIONAL | CLIENT: FIXED | VERIFIED BY: SR

Description. Let's analyze the effect of splitting minting a share of supply tokens $\Delta s = \Delta s_0 + \Delta s_1$ into two separate mints of shares Δs_0 and Δs_1. The costs of moving along the curve can be calculated as (not considering the errors introduced by precision changes from `WAD` to collateral decimals) for the combined mint:

$$C_{merged} = \max(1, \mathtt{cost}^{\uparrow}_{\mathcal{C}}(S + \Delta s) - \mathtt{cost}^{\downarrow}_{\mathcal{C}}(S))$$

and the cost for splitting the mint:

$$C_{split} = \underbrace{\max(1, \mathtt{cost}^{\uparrow}_{\mathcal{C}}(S + \Delta s_0) - \mathtt{cost}^{\downarrow}_{\mathcal{C}}(S))}_{C_0} + \underbrace{\max(1, \mathtt{cost}^{\uparrow}_{\mathcal{C}}(S + \Delta s_0 + \Delta s_1) - \mathtt{cost}^{\downarrow}_{\mathcal{C}}(S + \Delta s_0))}_{C_0}$$

$$\geq \max\left(2, \underbrace{\mathtt{cost}^{\uparrow}_{\mathcal{C}}(S + \Delta s_0) - \mathtt{cost}^{\downarrow}_{\mathcal{C}}(S) + \mathtt{cost}^{\uparrow}_{\mathcal{C}}(S + \Delta s_0 + \Delta s_1) - \mathtt{cost}^{\downarrow}_{\mathcal{C}}(S + \Delta s_0)}_{C^{inner}_{split}}\right)$$

If the values used above are above 1 so that the max functions would simplify, we would get:

$$C_{split} = C_{merged} + \underbrace{\left(\mathtt{cost}^{\uparrow}_{\mathcal{C}}(S + \Delta s_0) - \mathtt{cost}^{\downarrow}_{\mathcal{C}}(S + \Delta s_0)\right)}_{\Delta C(S + \Delta s_0)}$$

For the `PowerCurve` implementation we have:

$$\Delta C(s) = \left\lceil \frac{10^{18}}{\left\lfloor \frac{(10^{18}+c_1)c_2}{10^{18}} \right\rfloor} \texttt{pow}(s, 10^{18}+c_1) \right\rceil - \left\lfloor \frac{10^{18}}{\left\lceil \frac{(10^{18}+c_1)c_2}{10^{18}} \right\rceil} \texttt{pow}(s, 10^{18}+c_1) \right\rfloor$$

$$= \begin{cases} \in \{0,1\} & \text{if} \frac{(10^{18}+c_1)c_2}{10^{18}} \in \mathbb{Z} \\ \left\lceil \left\{ \frac{10^{18}}{\left\lfloor \frac{(10^{18}+c_1)c_2}{10^{18}} \right\rfloor + 1} \texttt{pow}(s, 10^{18}+c_1) \right\} + \frac{10^{18}}{\left\lfloor \frac{(10^{18}+c_1)c_2}{10^{18}} \right\rfloor^2 + \left\lfloor \frac{(10^{18}+c_1)c_2}{10^{18}} \right\rfloor} \texttt{pow}(s, 10^{18}+c_1) \right\rceil & \text{otherwise} \end{cases}$$

It is best to pick c_1 and c_2 such that:

$$\frac{(10^{18}+c_1)c_2}{10^{18}} \in \mathbb{Z} \tag{1}$$

This would guarantee that $\Delta C(S + \Delta s_0) \in \{0, 1\}$. Otherwise the value of $\Delta C(S + \Delta s_0)$ could grow, if $S + \Delta s_0$ gets high. For example when c_2 is a multiple of 10^{18} this condition is satisfied. Assume (1) is true then we get:

$$\begin{aligned} C_{merged} + \Delta C(S + \Delta s_0) &= \max(1 + \Delta C(S + \Delta s_0), C^{inner}_{split}) \\ &\leq \max(2, C^{inner}_{split}) \\ &\leq C_{split} \end{aligned}$$

One can show that if (1) is true then $C_{split} - C_{merge} \in \{0, 1, 2\}$

Recommendation. It is best to pick (c_1, c_2) curve parameters such that:

$$\frac{(10^{18}+c_1)c_2}{10^{18}} \in \mathbb{Z}$$

or instead of providing (c_1, c_2), one would provide $(c_1, c_3) \in \mathbb{N}^2$ for the curve such that the cost functions would be calculated using

$$\frac{10^{18}}{c_3} \texttt{pow}(s, 10^{18}+c_1)$$

and note that the provided c_3 value already would satisfy the required condition $c_3 \in \mathbb{Z}$ plus calculations would be simpler and more gas efficient. The omitted value of c_2 is only required for non-critical paths (view functions and guesstimations) to calculate the marginal price. This price can also be derived as the forward difference of the cost function.

Great set of values that can be chosen for c_3 could be of the form $2 \cdot 10^{18+d}$ where the d value should be high enough to squash any irregularities introduced by inner cost functions used such as `pow`.

Client. 2nd recommendation of using c_3 has been applied in `45db1fccd878a26b0da5ef814e5b6848f17387e2` .

I6. Profitability of sandwich attacks on mint or redeem.

Tags. OUT OF SCOPE | SEVERITY: INFORMATIONAL | CLIENT: ACKNOWLEDGED

Context.

- `PowerCurve.sol`

Description/Recommendation. Curve parameters should be chosen so that the profitability of sandwich attacks/strategies on mint or redeem can be estimated and potentially bounded (in an absence of redeem tax and treasury fees). For the set of parameters such as $(c_1, c_2) = (\lfloor \frac{n}{n+1} \rceil \cdot 10^{18}, 10^{6+18})$ and a convex almost quadratic cost function f (abusing the notations here to drop rounding directions and other error-introducing terms), one should try to bound:

$$\begin{aligned} &\underbrace{(f(x+y+z) - f(x+z))}_{\text{backrun redeem}} - \underbrace{(f(x+y) - f(x))}_{\text{frontrun mint}} \\ =\ &\underbrace{(f(x+y+z) - f(x+y))}_{\text{cost of minting } z \text{ at } x+y} - \underbrace{(f(x+z) - f(x))}_{\text{cost of minting } z \text{ at } x} \\ \overset{?}{\leq}\ &g(x, y, z) \end{aligned}$$

The above shows an adversary can extract the differences between minting costs at different outcome token supply shares.

Client. Acknowledged, we have started engaging with builders on bnb chain to see if we can try to avoid this issue altogether.

I7. Directed graph of chained markets.

Tags. OUT OF SCOPE | SEVERITY: INFORMATIONAL | CLIENT: ACKNOWLEDGED | VERIFIED BY: SR

Description. The set of markets and collateral tokens can create a directed forest where a direct edge between

- a non-outcome token collateral and a market or
- a market to market

indicate that the non-outcome token collateral or an outcome token of a market is used as collateral for the destination market of the edge. The roots of the trees in this forest are always non-outcome token collaterals. All these are currently enforced (or at least one should check to be enforced) through the causal effects of market deployments and non-zero initial seeding.

$$T_C \to M_0 \to M_1 \to \cdots \to M_n$$

- It is important that this directed graph would have no loops, and thus it should be a forest.
- Analysis would need to be performed on the downstream effect of an upstream market finalizing or resolving. For example if the upstream outcome token used is not a winner, all the downstream outcome tokens would basically have no inherent values (also related to backend and frontend operations)
- Effect of outcome tokens being used in secondary markets and a possibility of adding virtual edges to the directed forest using this secondary sources.

Client. Acknowledged. We fully intend to display this as a forest. As market and question creation is centralized, we have the ability to be selective with how the forest looks like - for e.g only tagging markets that will narrow in specificity. For example, the forest can look like

1. 'Will there be a match of A vs B?' -> 'Will A or B win?' -> 'How many points will A win by'
2. 'Will there be a match of A vs B?' -> 'Will A or B win?' -> 'How many points will B win by'

- Downstream OTs will not have inherent values if the upstream OT is not the winner. Only by successfully predicting across the entire chain of upstream OTs (parent markets) does the downstream OT have value, this is an intended effect of the market.
- OTs used in secondary markets (e.g a separate uniswap LP pool) will be priced differently from the primary market and naturally adds a virtual edge to the forest, leading to arbitrage opportunities. Fundamentally, even if the secondary market is a LMSR or LS-LMSR, it can still incur losses especially if the secondary market is not configured well. Secondary markets in which we do not own will not be displayed by us as we do not push users towards such pools.

I8. Convexity of cost functions.

Tags. OUT OF SCOPE | SEVERITY: INFORMATIONAL | CLIENT: ACKNOWLEDGED

Description. Given a cost function $f(x)$ (assuming no rounding and other error terms, no fess and no redeem tax), one would expect that the cost to mint at a higher total supply (x) should be more than the redeem collateral amount one would receive if the total supply was lower (z):

(The set A below can be $\mathbb{R}^+, \mathbb{Z}, \mathbb{N}, \cdots$)

$$\forall x, y, z \in A \text{s.t.} x \geq z \geq y \Rightarrow \underbrace{f(x+y) - f(x)}_{\text{mint } y \text{ at } x} \geq \underbrace{f(z) - f(z-y)}_{\text{redeem } y \text{ at } z}$$

Perhaps the above should also be desired when $x + y \geq z$. If so, we can also rewrite above as:

$$\forall x, y, z \in A \text{s.t.} x \geq z \Rightarrow \underbrace{f(x+y) - f(x)}_{\text{mint } y \text{ at } x} \geq \underbrace{f(z+y) - f(z)}_{\text{redeem } y \text{ at } z+y}$$

ie:

$$\forall y \in A \Rightarrow g(x) = f(x+y) - f(x) \text{is non-decreasing} \tag{1}$$

This would imply:

- discrete forward derivative of f with any step sizes is non-decreasing. The special case of step size 1 means $f(x+1) - f(x)$ is non-decreasing.
- For special sets of functions such as continuous functions and sets $A = \mathbb{R}^+$ one can show that f needs to be a convex function.
- For (left/right/both) differentiable functions f one can show that the derivative $f'(x)$ needs to be non-decreasing.

Recommendation. Provide a proof of (1) for the cost functions (including the correct rounding directions during mint/redeem) used in the codebase. For the power curve case, a simplified version would be to prove it for the `pow` function. If the cost functions used do not satisfy the above property, a replacement perhaps would need to be considered.

Client. Acknowledged.

Footnote. The codebase currently only checks the instantaneous version when $x = z$:

$$\underbrace{f^{\uparrow}(x+y) - f^{\downarrow}(x)}_{\text{mint } y \text{ at } x} \geq \underbrace{f^{\downarrow}(x+y) - f^{\uparrow}(x)}_{\text{redeem } y \text{ at } x+y}$$

I9. Constants used in `LogExpMath` do not conform to expected values.

Tags. OUT OF SCOPE | SEVERITY: INFORMATIONAL | CLIENT: ACKNOWLEDGED

Context.

- LogExpMath.sol#L59-L85

Description. One would expect that the constant provided in `LogExpMath.sol` conform to the following formula for:

$$a_i = \begin{cases} \left\lfloor e^{\frac{x_i}{10^{20}}} \cdot 10^{20} \right\rfloor & 2 \leq i \\ \left\lfloor e^{\frac{x_i}{10^{18}}} \right\rfloor & i \in \{0, 1\} \end{cases}$$

One could have also rounded-up above as long as a certain condition satisfied:

$$a_i \geq \bigotimes_{j<i} a_j \tag{1}$$

(the multiplication above needs to take into consideration the precision and also the order of the multiplication matters as it is not associative)

The current values of a_i do not conform to a consistent rounding direction and also the values of a_0 and a_1 are quite underestimated. Although even with these inconsistencies the current values still satisfy the property (1) which is important to show for example that `pow` is a non-decreasing function.

```
// 18 decimal constants
int256 constant x0 = 128000000000000000000; // ^27 000000000000000000
                   //  38877084059945950922226736883574780727281750630829988857729684183082856273 < 2 ^ 255 - 1
                   //  3887708405994595092222673688357478072728175063082998885772968418308285627 3.5772968418308285627 3
                   //  3887708405994595092222_267368835747807272817506308299888577296841830828562 73
                   //  3887708405994595092222_267368835747807272817506308299888577296841830828562 73
                   //  3887708405994595092222_267368835747807272817506308299888577296841830828562 73
                   //  3887708405994595092222_267368835747807272817506308299888577296841830828562 73
                   //  3887708405994595092222_267368835747807272817506308299888577296841830828562 73
int256 constant a0 = 38877084059945950922 2_000000000000000000000000000000000000; // ^e(x0) (no decimals) [BAD] [BAD] smaller
int256 constant x1 = 6400000000000000000; // ^26
                   //  62351490808116168829_09238708 floor(exp(64)) wolfram
                   //  62351490808116168829_09238708 sympy - floor
                   //  62351490808116168829_09238709 sympy - ceil / round
int256 constant a1 = 6235149080811616882 9_10000000; // ^e(x1) (no decimals) [BAD] [BAD] bigger

// 20 decimal constants
int256 constant x2 = 320000000000000000000; // ^25
                   //   7896296018268069516_097802263510822 floor / round
                   //   7896296018268069516_097802263510823 ceil
int256 constant a2 = 7896296018268069516_100000000000000; // ^e(x2) [BAD] [BAD]

int256 constant x3 = 160000000000000000000; // ^24
                   //   8886110520507872636 76_302374 floor /round
                   //   8886110520507872636 76_302375 ceil
int256 constant a3 = 888611052050787263676_000000; // ^e(x3) [BAD] [BAD]

int256 constant x4 = 80000000000000000000; // ^23
                   //   2980957987041728274 74_359 floor / round
                   //   2980957987041728274 74_360 ceil
int256 constant a4 = 29809579870417282747 4_000; // ^e(x4) [BAD] [BAD]

int256 constant x5 = 40000000000000000000; // ^22
                   //   5459815003314423907 81_1 floor / round
                   //   5459815003314423907 81_2 ceil
int256 constant a5 = 545981500331442390781_0; // ^e(x5) [BAD] [BAD]

int256 constant x6 = 20000000000000000000; // ^21
int256 constant a6 = 738905609893065022723; // ^e(x6) [GOOD] floor / round (ceil 7389056098930650227 24)

int256 constant x7 = 10000000000000000000; // ^20
int256 constant a7 = 271828182845904523536; // ^e(x7) [GOOD] floor / round (ceil 2718281828459045235 37)

int256 constant x8 = 5000000000000000000; // ^2-1
                   //   1648721270700128146 8_4 floor
                   //   1648721270700128146 8_5 ceil / round
int256 constant a8 = 164872127070012814 68_5; // ^e(x8) [BAD] ceil / round

int256 constant x9 = 2500000000000000000; // ^2-2
int256 constant a9 = 128402541668774148_407; // ^e(x9) [GOOD] floor / round (ceil 1284025416687741484 08)
                   //   128402541668774148_158 truncated 12-term taylor series
                   //   128402541668774148_029

int256 constant x10 = 1250000000000000000; // ^2-3
                   //    11331845306682631 68_2 floor
```

```
                //    1133148453066826316 8_3 ceil / round
int256 constant a10 = 1133148453066826316 8_3; // ^e(x10) [BAD] ceil / round

int256 constant x11 = 6250000000000000000; // ^2-4
int256 constant a11 = 106449445891785942956; // ^e(x11) [GOOD] floor / round (ceil 106449445891785942957)
```

Recommendation. More analysis is needed needed to make sure the constant used satisfy the potentially other required properties. The current set satisfies (1) even though it is not consistently calculated.

Client. Acknowledged.

Other Severity Issues

O1. The check regarding instant extraction of value in `_mintCollateralToOt` can be stronger.

Tags. LIKELIHOOD: HIGH CLIENT: FIXED VERIFIED BY: SR

Context.

- FTMarket.sol#L250-L255

Description. In `_mintCollateralToOt` , there is the following check:

```
// final check: mint cost >= redeem value
(uint256 collateralRedeemUserValue, uint256 collateralRedeemFee) =
market.curve.calRedeemValueByOtDelta(market.market, tokenId, otDeltaOut, dataSwap);
if ((collateralRedeemUserValue + collateralRedeemFee) > collateralDeltaIn) {
  revert Errors.MarketSwapPriceInvalidated(collateralDeltaIn, otDeltaOut);
}
```

to make sure, one cannot instantly extract value from the market. A stronger check would be to make sure one cannot instantly decrease the `totalMarketCap` A_{tot}. We have:

$$A_{tot} \xrightarrow{\text{mint}} A_{tot} + (\Delta a_c^{in} - \Delta a_c^{tr}) \overset{\text{redeem}}{\rightsquigarrow} A_{tot} + (\Delta a_c^{in} - \Delta a_c^{tr,0}) - (\Delta a_c^{out} + \Delta a_c^{tr,1})$$

Thus we get:

$$\Delta A_{tot} = (\Delta a_c^{in} - \Delta a_c^{tr,0}) - (\Delta a_c^{out} + \Delta a_c^{tr,1})$$

The check should enforce $\Delta A_{tot} \geq 0$, ie:

$$\Delta a_c^{in} - \Delta a_c^{tr,0} \geq \Delta a_c^{out} + \Delta a_c^{tr,1}$$

Recommendation. Implement the stronger check suggested above:

```
diff --git a/src/FTMarket.sol b/src/FTMarket.sol
index 41f4c30..b1d480b 100644
--- a/src/FTMarket.sol
+++ b/src/FTMarket.sol
@@ -250,7 +250,7 @@ contract FTMarket is FTERC6909, IFTMarket, TokenHelper, ReentrancyGuard, HasFTEv
         // final check: mint cost >= redeem value
         (uint256 collateralRedeemUserValue, uint256 collateralRedeemFee) =
             market.curve.calRedeemValueByOtDelta(market.market, tokenId, otDeltaOut, dataSwap);
-        if ((collateralRedeemUserValue + collateralRedeemFee) > collateralDeltaIn) {
+        if ((collateralRedeemUserValue + collateralRedeemFee) > (collateralDeltaIn - collateralToTreasury)) {
             revert Errors.MarketSwapPriceInvalidated(collateralDeltaIn, otDeltaOut);
         }
```

Client. Fixed in `ffcdcea376a40a8f0f888e5ab6405ee3984c90db` .

O2. `caller` input parameter can be removed from `_mintCollateralToOt` .

Tags. CLIENT: FIXED VERIFIED BY: SR

Context.

- FTMarket.sol#L232
- FTMarket.sol#L242
- FTMarket.sol#L245
- FTMarket.sol#L247
- FTMarket.sol#L94
- FTMarket.sol#L124

Description. The `caller` parameter passed to `_mintCollateralToOt` in all call-sites is the `msg.sender`. To optimize for gas, one can remove this parameter and inline the `msg.sender` value.

Note: `FTMarket` inherits from `FTERC6909` which inherits from `Context`. In `FTERC6909`, `_msgSender()` is used in place of `msg.sender` as this an internal function provided by `Context`. This pattern is broken in `FTMarket`.

Recommendation. Here is a rough patch that can be applied to avoid using the stack parameter `caller`:

```
diff --git a/src/FTMarket.sol b/src/FTMarket.sol
index 41f4c30..c96d0bd 100644
--- a/src/FTMarket.sol
+++ b/src/FTMarket.sol
@@ -91,7 +91,7 @@ contract FTMarket is FTERC6909, IFTMarket, TokenHelper, ReentrancyGuard, HasFTEv
         if (market.timestampEnd <= block.timestamp) revert Errors.MarketEnded();
         if (market.answer != 0 || market.isFinalised) revert Errors.MarketResolved();

-        _mintCollateralToOt(market, tokenId, otSeed, dataSwap, EMPTY_BYTES, msg.sender, market.treasury);
+        _mintCollateralToOt(market, tokenId, otSeed, dataSwap, EMPTY_BYTES, market.treasury);
     }

     /**
@@ -121,7 +121,7 @@ contract FTMarket is FTERC6909, IFTMarket, TokenHelper, ReentrancyGuard, HasFTEv
         if (market.answer != 0 || market.isFinalised) revert Errors.MarketResolved();
         if (receiver == address(this)) revert Errors.MarketReceiverIsMarket();

-        collateralIn = _mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap, dataCallback, msg.sender, receiver);
+        collateralIn = _mintCollateralToOt(market, tokenId, otDeltaOut, dataSwap, dataCallback, receiver);
     }

     /**
@@ -229,7 +229,6 @@ contract FTMarket is FTERC6909, IFTMarket, TokenHelper, ReentrancyGuard, HasFTEv
         uint256 otDeltaOut,
         bytes memory dataSwap,
         bytes memory dataCallback,
-        address caller,
         address receiver
     ) internal returns (uint256) {
         (uint256 collateralDeltaIn, uint256 collateralToTreasury) =
@@ -239,12 +238,12 @@ contract FTMarket is FTERC6909, IFTMarket, TokenHelper, ReentrancyGuard, HasFTEv

         _mint(receiver, tokenId, otDeltaOut);

-        emit MintSwap(caller, receiver, tokenId, collateralDeltaIn, otDeltaOut, collateralToTreasury);
+        emit MintSwap(_msgSender(), receiver, tokenId, collateralDeltaIn, otDeltaOut, collateralToTreasury);

         if (dataCallback.length > 0) {
-            IFTMintCallback(caller).onMint(collateralDeltaIn, otDeltaOut, dataCallback);
+            IFTMintCallback(_msgSender()).onMint(collateralDeltaIn, otDeltaOut, dataCallback);
         }
-        _transferIn(collateral, parentTokenId, caller, collateralDeltaIn);
+        _transferIn(collateral, parentTokenId, _msgSender(), collateralDeltaIn);
         _transferOut(collateral, parentTokenId, market.treasury, collateralToTreasury);

         // final check: mint cost >= redeem value
```

The above `patch` uses `_msgSender()` in place of `msg.sender`. Decide what pattern make sense to use for your use case and use it across the codebase.

Client. Fixed in `dd79affaebf8f65ae146cdda240e000d03d4ce55`.
